

Learning Objectives

Learners will be able to...

- Describe and implement tokenization and lemmatization on a corpus
- Describe and implement TFIDF and Cosine Similarity
- Create a chatbot in Python that uses an API to fetch data, with a tkinter GUI

info

Make Sure to Know

Intermediate Python.

Limitations

In this assignment, we only work with the tkinter and NLTK libraries.

Connecting to Github

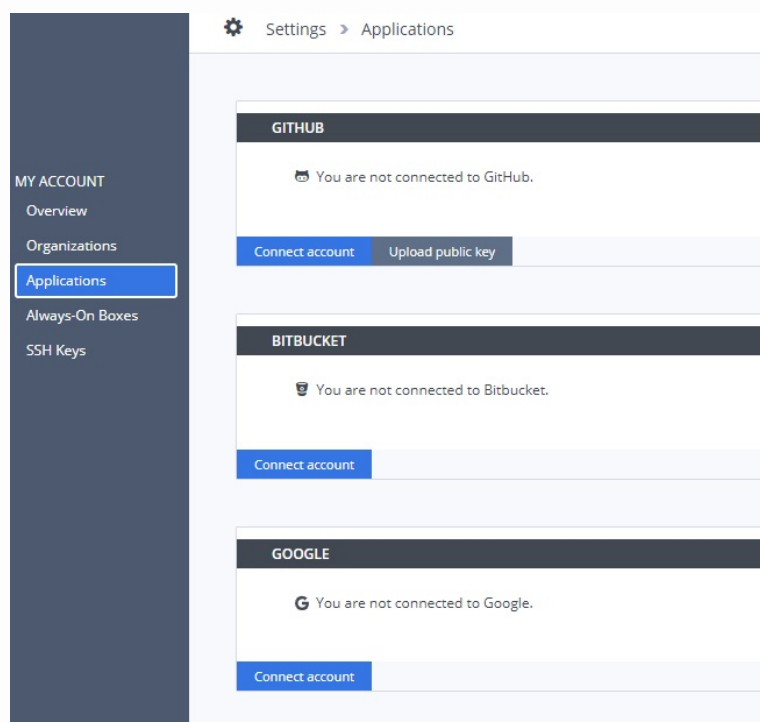
Connecting to GitHub

In this lab, you are going to build a Chatbot that uses an API. In order to showcase this project for your portfolio, you are going to use GitHub. If you do not yet have an account, please [create one](#) now. We are going to clone a repo that will contain the code for your Chatbot.

Connecting GitHub and Codio

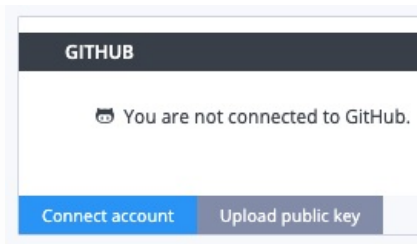
You need to connect GitHub to your Codio account. This only needs to be done one time.

- In your Codio account, click on your username
- Click on **Applications**



Codio Account

- Under GitHub, click on **Connect account**



connect account

- You will be using an SSH connection, so you need to click on **Upload public key**

Fork the Repo

- Go to the [nlp-apibot](#) repo. This repo is the starting point for your project.
- Click on the “Fork” button in the top-right corner.
- Click the green “Code” button.
- Copy the **SSH** information. It should look something like this:

```
git@github.com:<your_github_username>/nlp-apibot.git
```

info

Important

If you do not use the SSH information, you will have to provide your username and Personal Access Token (PAT) to GitHub each time you push or pull from the repository. See this [documentation](#) for setting up a PAT for your GitHub account.

In the Terminal

- Clone the repo. Your command should look something like this:

```
git clone git@github.com:<your_github_username>/nlp-apibot.git
```

- You should see a `nlp-apibot` directory appear in the file tree.

On the last page, we'll show you how to push to Github.

You are now ready for the lab!

Chatbot corpus and APIs

What is a corpus ?

- A corpus is a *large and structured set of text documents* that may serve as the **knowledge base** for a **corpus-based chatbot**.
- Such chatbots **don't have any manually created conditional rules** - instead, they use NLP techniques to learn mappings between user input and the required response.
- Depending on how it's designed, a corpus can be anywhere between **thousands** to **millions** of words!
- A chatbot corpus can be built/trained in a bunch of different ways, for instance:
 1. Manually copy-pasting data into a .txt file
 2. Using an API to fetch data from the web
 3. Using a library (for eg. *chatterbot*) to train a chatbot on a corpus
- The data used in any of these processes can be a **large conversational dataset** or even **news articles** and **online knowledge repositories** (eg. Wikipedia, etc.)

info

Corpus for this chatbot

- In this lab, we will use the **googlesearch** API with the python **requests** module to build custom corpora for our chatbot!

APIs, requests and googlesearch

- An application programming interface or API is a software interface for two applications or services to talk to each other.
- It is often used in the context of server-side HTTP-based web APIs, that have a defined **request-response** message system.
- HTTP is a system of rules for fetching, creating or editing resources on the web (for eg. HTML documents, databases, etc.).
- Here are a few common methods that HTTP provides:
 1. **GET**
The GET request is used to retrieve information using a given URL.
 2. **POST**
A POST request is used to send information to a server.
 3. **DELETE**
The DELETE method is used to request the server to delete a file at a location specified by the given URL.

info

The Python requests library

In this lab, we will use the Python requests library, that makes sending HTTP requests using Python extremely easy.

The googlesearch library

- The googlesearch Python library is used to do an effective Google search.
- It searches the given query string using Google, and returns a list of the URLs of top results.

info

Try this variation:

In the code file on the left, edit the query parameter to whatever you want to search on Google and click Try It button to see the results!

Now, let's start building our chatbot!

GUI for the Google Search Bot

info

chatbot_gui.py

We'll be using the same functions we used in The Simple Bot for the tkinter GUI and keep them in a separate file called `chatbot_gui.py`. You should be able to see that file in the top left pane going forward.

(Click on the object/function description to see it highlighted in the code file)

- `baseWindow` -
- `chatWindow` -
- `userEntryBox` -
- `sendButton` -
- `send()` -
- `create_and_insert_user_frame()` -
- `create_and_insert_bot_frame()` -

Fetching the top Google result from user input

Now that we're all set for the GUI, let's start working on the logic for our Chatbot!

To start with, we want to build a corpus for our chatbot for every `user_input` by fetching the top Google search result.

info

Writing Code

To your left, you should see an opened file called `api_bot.py`. Use this file and reference the syntax below to work on this Lab.

Let's import all the Python packages we'll be using in this lab

Paste the following code in the left pane:

```
import string
import nltk
import requests

from googlesearch import search
from lxml import html
from bs4 import BeautifulSoup
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from nltk.stem import WordNetLemmatizer
```

Now, let's write a function called `get_bot_response` that takes in the `user_input`, and returns the `bot_response`

To get the text from the top Google search result:

- Use the `googlesearch` library to run the search with the `user_input`
- With the Python `requests` library, get the webpage of the top search result
- We'll extract the *html* elements by using the `BeautifulSoup` library and the `lxml` *html* parser and generate the text corpus

Paste the following code in the left pane, right under the import statements:

```
# helper function to generate text corpus from html elements
def generate_corpus(all_p_elements):
    corpus = ""
    for p_element in all_p_elements:
        corpus += '\n' + ''.join(p_element.findAll(text = True))
    return corpus

def get_bot_response(user_input):
    # default bot response
    bot_response = "I'm sorry, I don't think I can help you with that :("
    try:
        # use the google search api to fetch top 3 search results
        google_search_results = list(search(user_input, stop=3, pause=1))

        # use the requests api to fetch the top result webpage
        webpage = requests.get(google_search_results[0])
        webpage_tree = html.fromstring(webpage.content)
        webpage_soup = BeautifulSoup(webpage.content, "lxml")

        # extract all <p> elements from webpage soup object
        all_p_list = webpage_soup.findAll('p')

        # generate corpus from all <p> elements
        google_search_corpus = generate_corpus(all_p_list)

    except:
        # return the default response if corpus is empty
        if len(google_search_corpus) == 0:
            return bot_response
```

▼ Code

Your code should look like this:

```

import string
import nltk
import requests

from googlesearch import search
from lxml import html
from bs4 import BeautifulSoup
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from nltk.stem import WordNetLemmatizer

# helper function to generate text corpus from html elements
def generate_corpus(all_p_elements):
    corpus = ""
    for p_element in all_p_elements:
        corpus += '\n' + ''.join(p_element.findAll(text = True))
    return corpus

def get_bot_response(user_input):
    # default bot response
    bot_response = "I'm sorry, I don't think I can help you with that :("

    try:
        # use the google search api to fetch top 3 search results
        google_search_results = list(search(user_input, stop=3, pause=1))

        # use the requests api to fetch the top result webpage
        webpage = requests.get(google_search_results[0])
        webpage_tree = html.fromstring(webpage.content)
        webpage_soup = BeautifulSoup(webpage.content, "lxml")

        # extract all <p> elements from webpage soup object
        all_p_list = webpage_soup.findAll('p')

        # generate corpus from all <p> elements
        google_search_corpus = generate_corpus(all_p_list)

    except:
        # return the default response if corpus is empty
        if len(google_search_corpus) == 0:
            return bot_response

```

And that's it! We've successfully written the logic that gets us the required data!

Recap: Text preprocessing, TFIDF and Cosine Similarity

In the next part, we want to process this text to find the most similar sentence to each user input.

At this point, you must be familiar with some of the NLP concepts that we'll use in this lab. Let's review them:

1. Text preprocessing

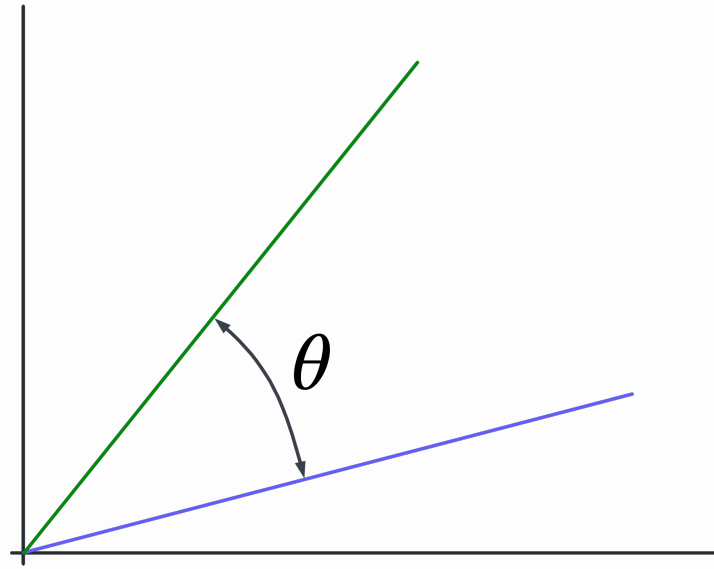
- Tokenization takes raw text and generates a list of word or sentence tokens.
- Lemmatization reduces words to their most basic meaning.
- It will give us a word that represents a grouping of words (e.g., the lemma of the word "every" is still "every").

2. TF-IDF

- Term Frequency-Inverse Document Frequency (TF-IDF) is sometimes referred to as "weighted" bag of words.
- TF-IDF rescales the frequency of words by how often they appear in all documents, so that the scores for frequent words like "the" that are also frequent across all documents are penalized.

3. Cosine Similarity

- If Euclidean distance is the straight line distance between vectors, Cosine similarity is the angular distance between vectors:



.guides/img/cosine

The formula is:

$$\cos(\theta) = \frac{A \cdot B}{\|x\| * \|y\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

Wrapping up the get_bot_response function

Now that we've reviewed some NLP concepts, let's implement them in our chatbot!

- First, we will tokenize the text by using `nltk` to get sentence tokens.
- This will be our input to the `sklearn` TF-IDF Vectorizer, to generate TF-IDF scores of our `user_input` and all the sentences.
- This Vectorizer uses a tokenizer, which we'll implement by using the `WordNetLemmatizer`.
- Now, the only thing left is to calculate the Cosine Similarity and choose the sentence with the highest score.

This will be the sentence that will be returned as the `bot_response`

Let's see how that looks like in code. Paste the following code in the top left panel by replacing the `get_bot_response` function

```
# preprocessing (lemmatization, removing punctuation)
def LemTokens(tokens):
    lemmatizer = WordNetLemmatizer()
    return [lemmatizer.lemmatize(token) for token in tokens]

def LemNormalize(text):
    remove_punct_dict = dict((ord(punct), None) for punct in
                             string.punctuation)
    return LemTokens(nltk.word_tokenize(text.lower().translate(remove_punct_dict)))

def calculate_cosine_similarity(tfidf):

    # Calculate cosine_similarity
    cosine_sim_matrix = cosine_similarity(tfidf[-1], tfidf)

    # get index second highest value for cosine_similarity
    # highest value will be the user_input itself
    idx = cosine_sim_matrix.argsort()[0][-2]
    cosine_sim_flattened = cosine_sim_matrix.flatten()
    cosine_sim_flattened.sort()
    req_tfidf = cosine_sim_flattened[-2]

    return idx, req_tfidf
```

```

def get_bot_response(user_input):
    # default bot response
    bot_response = "I'm sorry, I don't think I can help you with that :("

    try:
        # use the google search api to fetch top 3 search results
        google_search_results = list(search(user_input, stop=3, pause=1))

        # use the requests api to fetch the top result webpage
        webpage = requests.get(google_search_results[0])
        webpage_tree = html.fromstring(webpage.content)
        webpage_soup = BeautifulSoup(webpage.content, "lxml")

        # extract all <p> elements from webpage soup object
        all_p_list = webpage_soup.findAll('p')

        # generate corpus from all <p> elements
        google_search_corpus = generate_corpus(all_p_list)

        # Tokenisation
        sentence_tokens =
        nltk.sent_tokenize(google_search_corpus) # converts raw
        text to list of sentences

        # Calculate TFIDF matrix
        sentence_tokens.append(user_input)
        tfidf_vectorizer =
        TfidfVectorizer(tokenizer=LemNormalize,
        stop_words='english')
        tfidf = tfidf_vectorizer.fit_transform(sentence_tokens)

        idx, req_tfidf = calculate_cosine_similarity(tfidf)

        if(req_tfidf==0):
            # if value of cosine_similarity == 0, similar
            sentence not found
            bot_response = "I am sorry! I don't think I can help
            you with that at the moment..."
        else:
            bot_response = sentence_tokens[idx]
            sentence_tokens.remove(user_input)
        return bot_response

    except:
        # return the default response if corpus is empty
        if len(google_search_corpus) == 0:
            return bot_response

```

▼ Code

Your code should look like this:

```

import string
import nltk
import requests

from googlesearch import search
from lxml import html
from bs4 import BeautifulSoup
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from nltk.stem import WordNetLemmatizer

# helper function to generate text corpus from html elements
def generate_corpus(all_p_elements):
    corpus = ""
    for p_element in all_p_elements:
        corpus += '\n' + ''.join(p_element.findAll(text = True))
    return corpus

# preprocessing (lemmatization, removing punctuation)
def LemTokens(tokens):
    lemmatizer = WordNetLemmatizer()
    return [lemmatizer.lemmatize(token) for token in tokens]

def LemNormalize(text):
    remove_punct_dict = dict((ord(punct), None) for punct in
                             string.punctuation)
    return
        LemTokens(nltk.word_tokenize(text.lower()).translate(remove_punct_dict))

def calculate_cosine_similarity(tfidf):

    # Calculate cosine_similarity
    cosine_sim_matrix = cosine_similarity(tfidf[-1], tfidf)

    # get index second highest value for cosine_similarity
    # highest value will be the user_input itself
    idx = cosine_sim_matrix.argsort()[0][-2]
    cosine_sim_flattened = cosine_sim_matrix.flatten()
    cosine_sim_flattened.sort()
    req_tfidf = cosine_sim_flattened[-2]

    return idx, req_tfidf

def get_bot_response(user_input):
    # default bot response

```



```

bot_response = "I'm sorry, I don't think I can help you with
that :("
try:
    # use the google search api to fetch top 3 search
    results
    google_search_results = list(search(user_input, stop=3,
    pause=1))

    # use the requests api to fetch the top result webpage
    webpage = requests.get(google_search_results[0])
    webpage_tree = html.fromstring(webpage.content)
    webpage_soup = BeautifulSoup(webpage.content, "lxml")

    # extract all <p> elements from webpage soup object
    all_p_list = webpage_soup.findAll('p')

    # generate corpus from all <p> elements
    google_search_corpus = generate_corpus(all_p_list)

    # Tokenisation
    sentence_tokens =
    nltk.sent_tokenize(google_search_corpus) # converts raw
    text to list of sentences

    # Calculate TFIDF matrix
    sentence_tokens.append(user_input)
    tfidf_vectorizer =
    TfidfVectorizer(tokenizer=LemNormalize,
    stop_words='english')
    tfidf = tfidf_vectorizer.fit_transform(sentence_tokens)

    idx, req_tfidf = calculate_cosine_similarity(tfidf)

    if(req_tfidf==0):
        # if value of cosine_similarity == 0, similar
        sentence not found
        bot_response = "I am sorry! I don't think I can help
        you with that at the moment..."
    else:
        bot_response = sentence_tokens[idx]
        sentence_tokens.remove(user_input)
    return bot_response

except:
    # return the default response if corpus is empty
    if len(google_search_corpus) == 0:
        return bot_response

```

That was the last step! We have successfully built our Google Search Bot!
Let's run it and interact with it!

Pushing to Github

Pushing to GitHub

Before continuing, you must push your work to GitHub.
In the terminal:

- Commit your changes:

```
git add .  
git commit -m "Finished The API Bot"
```

- Push to GitHub:

```
git push
```